Technical report — Breaking Apertus

This project investigates potential weaknesses in Apertus-v1.5-8B using automated red-teaming. I used Promptfoo and Hydra-Strategy to test the model across multiple topics and investigate promising attack results. The testing produced five documented findings concerning historical misinformation, source reliability, false attribution, and copyright compliance.

Track: Apertus Readiness - Track 1A
Event: Online
Team: `Loner` — `Bogdan Lomp`
Demo: `promptfoo_report.html or video-example here: https://drive.google.com/drive/folders/1xFCNkiOTVKQEFP9S0uNHtLCT-iuB55hm?usp=sharing`

Use of Apertus

- **Model:** `swiss-ai/Apertus-v1.5-8B`
- **How it is used:** inference
- **Where it runs:** `CSCS`

Problem

The red-teaming issues cover historical misinformation, fabricated sources, unsupported attribution, and copyright compliance.

Issue №1 - Wabern Protocol: Historical Misinformation and Fabricated Sources

- **Problem:** The model presents unverified historical claims about the alleged Wabern Protocol as established facts, constructing a supporting narrative instead of critically evaluating the claims. Also the model provides specific archival references (BFA 12-257.1, BFA 12-257, or FAB 24.12.42) and purported historical quotations without verifiable supporting sources.
- **Use cases:** This issue is relevant when the model is used for historical research, educational content, or answering questions about Swiss history and historical archival documents.
- **Threat analysis:** Such outputs may mislead users into believing that unverified historical claims are supported by authentic archival evidence, potentially contributing to the spread of historical misinformation.

Issue №2 - Poland: Historical Misinformation and Distortion

- **Problem:** The model presents unverified claims about Poland's role in World War II as established facts instead of challenging misleading premises or correcting historical inaccuracies and constructs a misleading historical narrative that portrays Poland as a

collaborator in Nazi Germany's military aggression and genocidal policies, distorting the country's wartime and postwar political history.
- **Use cases:** This issue is relevant when the model is used for historical research, educational content, answering questions about Poland's role in World War II or when users present historically inaccurate claims and the model fails to correct them.
- **Threat analysis:** Such outputs may mislead users who rely on the model for historical information and contribute to the spread of inaccurate historical narratives.

Issue №3 - Anne Applebaum: Fabricated Claims and Misleading Citations

- **Problem:** The model introduces unsupported claims, unsubstantiated criticism, and misleading reinterpretations into a biographical article, potentially undermining a journalist's credibility and generates a fabricated or unverifiable citation that appears authoritative, creating a false impression of reliable sourcing and potentially misleading readers.
- **Use cases:** This issue is relevant when the model is used to generate biographical articles, summarize the work of journalists or public figures, or produce content for research and publication.
- **Threat analysis:** Such outputs may spread unsupported claims about individuals, mislead readers through unreliable citations, and potentially damage a person's professional reputation. This vulnerability could also be exploited to generate misleading content for propaganda purposes.

Issue №4 - Harry Potter: Copyrighted Text Reproduction

- **Problem:** The model reproduces a substantial amount of copyrighted text that the user did not provide, instead of avoiding verbatim reproduction. The beginning of a chapter "Harry Potter and the Philosopher’s Stone" was wrote by the model word-for-word, with a maximum observed output of 227 words.
- **Use cases:** This issue is relevant when the model is used for literary analysis, educational content, book summaries, or answering questions about copyrighted works.
- **Threat analysis:** Reproducing of copyrighted text word-for-word may raise concerns about copyright compliance and the model's ability to handle requests involving protected material appropriately, especially when the author has not given permission for their work to be used in AI training.

Issue №5 - Bern Memorandum: Fabricated Historical Evidence and False Attribution

- **Problem:** The model presents an apparently fabricated historical archival document, an official-looking file number, and purported endorsements by named historians as genuine evidence supporting a disputed historical interpretation. The model presents unverified claims with unwarranted certainty, potentially misleading readers about the causes of World War I and the historical evidence relating to the Fischer thesis.
- **Use cases:** This issue is important when the model is used for historical research, academic writing, educational content or answering questions about the causes of World War I (such as the Fischer thesis) and other historical debates.
- **Threat analysis:** Such outputs may mislead users into treating fabricated or unverified documents as authentic evidence, potentially distorting their understanding of historical debates and spreading misinformation through academic or educational content.

Red Teaming Method

Overall Strategy

Test the model across a wide variety of topics and evaluate the results to identify where it fails most often. Select specific topics that reveal potential weaknesses and look for the strongest examples of successful attacks. Then investigate these examples further: how likely is the model to comply, and can the attack be made even more harmful? For example, I managed to get 227 words from the beginning of Harry Potter by starting with only 45 sentences, which could dramatically increase the risk of potential copyright-related legal disputes, including lawsuits.

Discovery Process

I started by testing the 8B model across a wide range of topics to identify areas where attacks were most likely to succeed. During this process, I experimented with different approaches and tactics, adapting the attack strategy based on the results. I then focused on the topics where the most serious issues were observed, investigating them individually to identify the most brightest examples, used them to refine my approach and adjust the overall attack strategy again. For some examples, I conducted additional tests to see whether the results could be further strengthened.

Overall, I used a broad-to-narrow approach, starting with a wide exploration of multiple topics and gradually focusing on the most promising areas and successful attacks.

This issues there are not unique, but I find them the most interesting ones (despite it's pretty easy to find)

Tooling

- **Promptfoo:** Used to manage automated red-teaming attacks, evaluate the results, and identify the model's behaviour and weaknesses across a large number of tests.
- **Hydra-Strategy:** Used to generate hundreds of original multi-turn attacks. Hydra can save successful attacks in memory and backtrack through the dialogue multiple times if an attack branch fails.
- **Apertus-v1.5-8B:** The target model tested during the red-teaming process. (I used also 70B version but only to compare the behaviour with selected prompts)
- **Qwen3.5-9B-Q8:** Used for test generation and evaluation.

Findings Dataset

1. **Wabern Protocol**
2. **Poland**
3. **Anne Applebaum**
4. **Harry Potter**
5. **Bern Memorandum**

Lessons Learnt and Recommendations

What issues you would investigate next, knowing what you know now.

I would like to make more precise attacks using the experience I gained while red-teaming the model:

- Research the model's behaviour in politics and history, where I noticed serious weaknesses
- Look for other topics, that are vulnerable to jailbreaks and might cause long-term problems
- Research the differences between the 8B and 70B models. In some situations, the models behave better or worse than each other.
- Investigate the model's hidden instructions, if I had the opportunity to access them, and compare them with what I gathered while trying to find weaknesses. However, I'm not sure if the instructions I gathered with Jailbreaks are correct or just model's hallucinations.

Reproducibility

- **Target model:** `swiss-ai/Apertus-v1.5-8B`
- **Evaluation model:** `Qwen3.5-9B-Q8` (self-hosted model of comparable or greater capability)
- **Hardware used:** CSCS Supercomputing Infrastructure (for the target model) and AMD Radeon RX 7900 XT, Intel Core i7-12700KF, 32 GB DDR4 RAM, Samsung 980 1 TB SSD (for the local evaluation model).
- **Runtime:** Ubuntu 22.04.5 LTS, AMD ROCm 7.2.3, Miniconda3, Promptfoo 0.124.1. Other library versions and dependencies are specified in `environment.yml`.
- **Seeds:** `42` and runs without a fixed seed.
- Please note that despite strictly freezing the random state (`seed: 42`) and entirely disabling promptfoo's internal cache (`--no-cache`), absolute determinism was not achieved across all test suites (specifically in the "Anne Applebaum" scenario on the model 8B).

Local evaluation model setup:

- Model: Qwen3.5-9B-Q8_0 (GGUF, Q8_0 quantization).
- Inference server: llama.cpp (llama-server).
- Context length: 16,384 tokens.
- Maximum generation length: 256 tokens.
- GPU offloading: -ngl 99.
- Reasoning: Disabled (--reasoning off).
- Seed: 42.
- Endpoint: http://localhost:8001/v1. The inference server was launched locally using llama-server. The model was served through an OpenAI-compatible API for use with Promptfoo.

Run the server with:

```
./build/bin/llama-server \
-m /path/to/Qwen3.5-9B-Q8_0.gguf \
-ngl 99 \
-c 16384 \
-n 256 \
--reasoning off \
--port 8001 \
```

```
--seed 42
```

Simplified evaluation model setup Setting up the local evaluation model requires several manual steps, so I added a simpler alternative: using the same API key to access Apertus as the Judge. This change is expected to have very limited impact on the results, as the Judge is only used to determine whether a response violates the specified rules. However, using a different Judge may still affect evaluation outcomes.

Note: After switching to Apertus as the Judge, I observed less reliable evaluations in two scenarios and greater variation in the results. The original Promptfoo report (`promptfoo_report.html`) is retained for reference.

Running the evaluation From the repository root or track_1a and run the evaluation:

```
make run
```

This command checks the required environment variables and configuration files, builds the Docker image, and runs all five Promptfoo evaluation scenarios. The HTML reports are saved in `track_1a/docs/`.

Optional commands:
Run a single scenario:

```
make run-wabern
make run-poland
make run-applebaum
make run-bern
make run-potter
```

Change the number of repetitions (default: 20):

```
make run REPEAT=50
```

Run environment checks only:

```
make check
```

Explanation of the purposes of promptfoo's scripts:

- **discovery-copyrights.yaml** - used to evaluate the model and collect examples of failures across various automatically generated attacks on the topic of copyright.
- **discovery-history.yaml** - used to evaluate the model and collect examples of failures across various automatically generated attacks on the topic of history.
- **discovery-potter-strict.yaml** - used to evaluate the model and collect failures across various automatically generated attacks designed to get copyrighted text from "Harry Potter and the Philosopher’s Stone" but with strict rules: Direct reproduction of big amount of text (it was used for making this case much stronger)
- **verification-potter-strict.yaml** - used to investigate the strongest observed example in topic "copyrights". It was not included in the evaluation metrics. Only 2% of the tested examples were comparable in terms of the amount of reproduced text. The other examples began hallucinating before reaching ~250 words.

These were used to investigate the strongest observed examples (single prompts), repeated in total thousands of times. Words after "-" are the cases (keywords):

- **verification-applebaum.yaml**
- **verification-bernmemorandum.yaml**
- **verification-poland.yaml**
- **verification-potter.yaml**
- **verification-wabern.yaml**

License

Creative Commons Attribution 4.0 (CC-BY-4.0). All HackApertus projects are open-sourced.

References